

Personal repository edition

Updated 20 September 2026: pre-training managed identities and container registry permissions.

Source: https://github.com/alvinea28/own-ml-ops-v2/blob/main/documentation/deployguides/deployguide_ado.md

Deployment Guide - Azure DevOps

This document will guide you through using the MLOps V2 project generator to deploy a single-environment ("Prod") demo project using only Azure DevOps to host source repositories and pipelines. See notes at the end for guidance on multi-environment MLOps and adapting the pattern to your use case.

Personal repository edition: Use [alvinea28/own-ml-ops-v2](#) on **main**, not the original Azure accelerator. The default classical / AML CLI v2 / Bicep path uses the repaired [taxi project](#). See [the personal setup notes](#) for prerequisites and differences from the upstream examples. Screenshots retain their original example labels; use the repository names and parameters in the updated text below.

What is imported, and what does the initializer create?

The imported taxi repository is a reusable source template, not the new project. The initializer populates a separate target repository with the working code and creates its pipeline definitions. It does not create the Azure Repos repository itself; create that empty target before running the initialization YAML.

Role	Azure Repos repository	How it is created
SOURCE: repaired template	<code>taxi-fare-regression-template</code>	Import alvinea28/taxi-fare-regression from GitHub once. It contains the reusable working fixes.
TARGET: your new project	For example, <code>taxi-fare-regression-demo</code>	Create a new empty Azure Repos repository. The initializer copies the source files into it, commits them, and pushes them. Do not import the taxi template into this target.

Run the initializer with `taxiTemplateRepoName: taxi-fare-regression-template` and `repoName: taxi-fare-regression-demo` (or your own new target name). Never select your existing working taxi project or the source template as the target.

Which repositories are actually in the personal GitHub account? Only [alvinea28/own-ml-ops-v2](#) and [alvinea28/taxi-fare-regression](#) were created for this workflow. The two upstream companion repositories have **not** been copied into that account:

- [Azure/ml-ops-templates](#) remains a **required** shared dependency for the generated training and deployment pipelines.
- [Azure/ml-ops-project-template](#) is **not used by the simplified taxi initializer**. It is only needed for the separate [advanced initializer](#), which supports the original upstream scenarios.

Import sources **from GitHub into Azure Repos**. Even if you later create personal copies of the companions, the current YAML still needs Azure Repos imports because it uses Azure Repos `git://` checkouts, not direct GitHub checkouts. The import table below intentionally keeps the real upstream URLs for dependencies that were not copied.

[PDF version of this guide](#)

Import the taxi regression template

Complete this source import **before** running the initializer. The GitHub repository is named `taxi-fare-regression`; its Azure Repos copy must use the separate source name `taxi-fare-regression-template`.

1. Open your Azure DevOps project, for example **fixed-mlops-v2**.
2. Select **Repos** → **Files**, open the repository dropdown, and select **Import repository**. Create a separate source repository through this dialog; do not import into your empty `taxi-fare-regression-demo` destination.
3. Enter these values:

Import field	Value
Repository type	Git
Clone URL	https://github.com/alvinea28/taxi-fare-regression.git
Name / New repository name	taxi-fare-regression-template

4. Click **Import** and wait for the import to finish. The GitHub source is public at the time of this update. If it is private when you import, authenticate with an account that has read access; enter credentials only in the secure import dialog, never in YAML, source files, or chat.
5. Open **taxi-fare-regression-template**, select **main**, and confirm that `template-manifest.json`, `data-science`, `mlops`, and `infrastructure` are present. A repository containing only an initial README is not a completed source import.
6. Keep this imported source repository. Separately create your empty destination with **Add a README** selected on **main**, then run the normal initializer:

Initializer input	Example value
Azure DevOps project	fixed-mlops-v2
New taxi project repository	taxi-fare-regression-demo
Source template repository	taxi-fare-regression-template

If Azure DevOps asks for repository authorization, use **View/Permit** for this initializer. Ensure the project Build Service can read the source and contribute to the new destination. A recreated repository has a new identity even when its name is unchanged, so previous authorizations may need to be granted again.

The `-template` suffix belongs to the Azure Repos name, not the GitHub URL. The initializer copies from the source into the empty destination. When retesting, keep the source template and use a new empty destination; do not delete the source or your original working taxi repository.

Check the repositories before running initialization

Typing a repository name into **Run pipeline** does not create or import it. In the Azure DevOps project selected by `adoProjectName`, check **Repos** first:

Repository	Required state before initialization
Your accelerator repository	Latest personal accelerator <code>main</code> , including the three-input initialization YAML
<code>taxi-fare-regression-template</code>	Imported repaired source on <code>main</code> , with <code>template-manifest.json</code> , <code>data-science</code> , <code>mlops</code> , and <code>infrastructure</code> present
<code>mlops-templates</code>	Imported shared helpers on <code>main</code> , used by the generated deployment/training pipelines
<code>taxi-fare-regression-demo</code>	A separately created, empty target; a single initial README commit on <code>main</code> is supported

The upstream `mlops-project-template` repository is **not a substitute** for `taxi-fare-regression-template`. Importing the generic upstream repository does not create the repaired source under the other name.

If you imported the repaired source under a different Azure Repos name, enter that exact existing name as `taxiTemplateRepoName`. Keep it separate from the target and your original working repository. A GitHub repository existing in your account does not mean its Azure Repos copy exists.

Prerequisites:

- One or more Azure subscription(s) based on whether you are deploying Prod only or Prod and Dev environments
 - **Important:** - As mentioned in the **Prerequisites** at the beginning [here](#), if you plan to use either a Free/Trial or similar learning purpose subscriptions, they might pose 'Usage + quotas' limitations in the default Azure region being used for deployment. Please read provided instructions carefully to successfully execute this deployment.
- An Azure DevOps organization
- Ability to create Azure service principals to access / create Azure resources from Azure DevOps
- If using Terraform to create and manage infrastructure from Azure DevOps:
 - Install the [Terraform extension for Azure DevOps](#)
 - **This modernized version requires Terraform v1.10.0 or higher** with azurerm provider ~> 4.52.0
- **Python 3.11 or higher** for local development and testing

Steps to Deploy

1. Clone and Configure the MLOps V2 Solution Accelerator

Create a copy of the MLOps V2 Solution Accelerator in your organization that can be used to bootstrap new ML projects.

2. Create and Configure a New ML Project Repo

Use the solution accelerator to create a new ML project according to your scenario and environments and configure it for deployment.

3. Deploy and Execute Azure Machine Learning Pipelines

Run Azure DevOps pipelines in your new project to deploy Azure Machine Learning infrastructure, deploy and run a training pipeline, and a deployment pipeline.

Clone and Configure the MLOps V2 Solution Accelerator

This section guides you through creating an Azure DevOps project to contain the MLOps repositories and your ML projects, importing the MLOps repositories, and configuring the project with permissions to create new pipelines in the ML projects you generate.

Below are the three repositories to import for the repaired taxi scenario. They serve as the "project factory" for classical / AML CLI v2 / Bicep projects. The simplified taxi initializer does not use the upstream project-template repository.

Repository	Azure Repos name	Role
https://github.com/alvinea28/own-ml-ops-v2	<code>mlops-v2</code> in this walkthrough, or <code>own-ml-ops-v2</code> if you keep the GitHub name	Personal accelerator with the repaired initializer on <code>main</code> .
https://github.com/alvinea28/taxi-fare-regression	<code>taxi-fare-regression-template</code>	Repaired standalone taxi project; use a different repository for the generated application.
https://github.com/Azure/ml-ops-templates	<code>mlops-templates</code>	Required shared Azure ML pipeline helpers; this remains an upstream dependency.

Only if you need the separate [advanced initializer](#) for CV, NLP, Python SDK, Responsible AI, Terraform, or original upstream templates, import <https://github.com/Azure/mlops-project-template> as `mlops-project-template`. Do not import it for this taxi walkthrough, and do not substitute the standalone taxi repository for the advanced initializer's nested template layout.

Visibility is configured separately for each GitHub repository. Both personal repositories are public at the time of this update. If a source is private when you import, authenticate in the Azure DevOps UI with an account that has read access; never store an import token in source files. Imports are one-time copies, not automatic mirrors.

1. Navigate to [Azure DevOps](#) and the organization where you want to create the project. [Create a new organization](#) for your project, if needed.
2. Create a new project named `mlops-v2`.

Create new project



Project name *

mlops-v2



Description

Visibility



Public

Anyone on the internet can view the project. Certain features like TFVC are not supported.



Enterprise

[Members of your enterprise](#) can view the project.



Private

Only people you give access to will be able to view this project.



Advanced

Cancel

Create

3. Import the MLOps V2 repositories. In your new `mlops-v2` project, select the Repos section on the left side.



The default repo, `mlops-v2`, is empty. Under "Import a repository", select **Import**.

Import a repository

Import

Enter <https://github.com/alvinea28/own-ml-ops-v2> into the Clone URL field. Click import at the bottom of the page. This imports the personal accelerator into the Azure Repos repository named `mlops-v2` in this walkthrough.

Import a Git repository



Repository type

 Git 

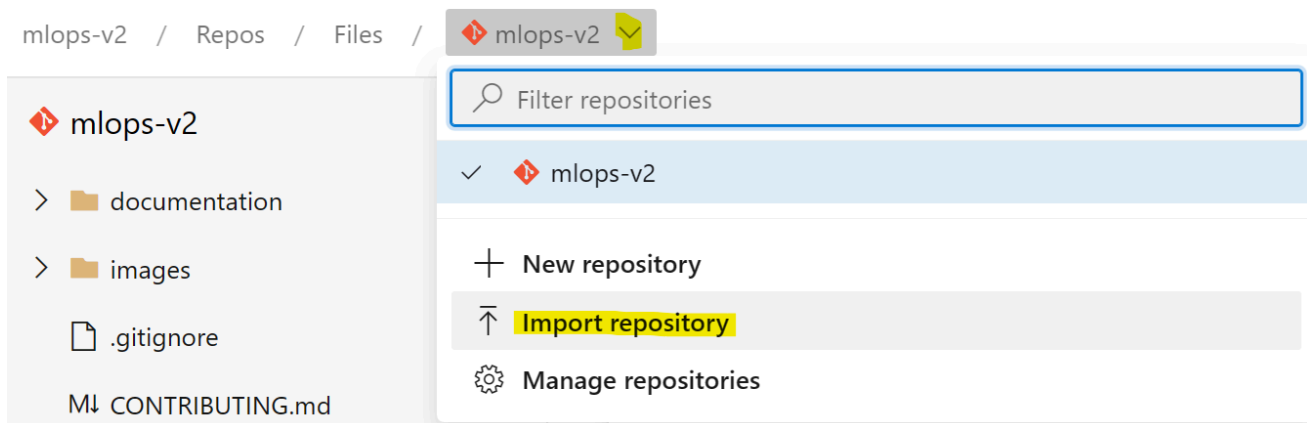
Clone URL *

<https://github.com/Azure/mlops-v2>

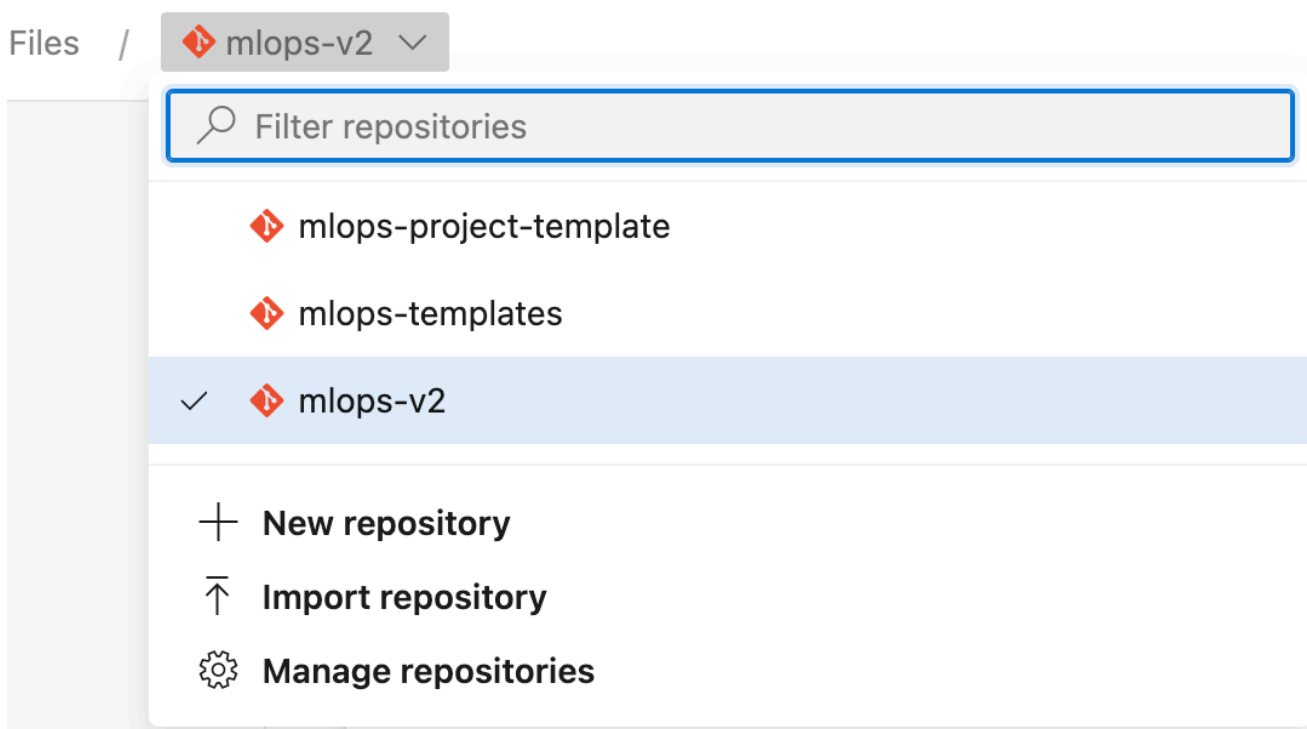
☐ Requires Authentication

At the top of the page, open the Repos drop-down and repeat the import for:

- <https://github.com/alvinea28/taxi-fare-regression.git>, naming its Azure Repos copy **taxi-fare-regression-template**. Follow [Import the taxi regression template](#). This is the reusable **source**, not the new application target.
- <https://github.com/Azure/mlops-templates>, naming its Azure Repos copy **mlops-templates**. This required helper repository has not been copied to the personal GitHub account.
- Only for the separate advanced initializer: <https://github.com/Azure/mlops-project-template> as **mlops-project-template**. It is not needed or used by the simplified taxi initializer and has not been copied to the personal GitHub account.



When done, you should see [all three MLOps V2 repositories](#) in your project.

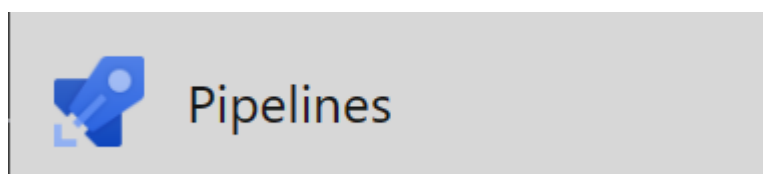


Important:

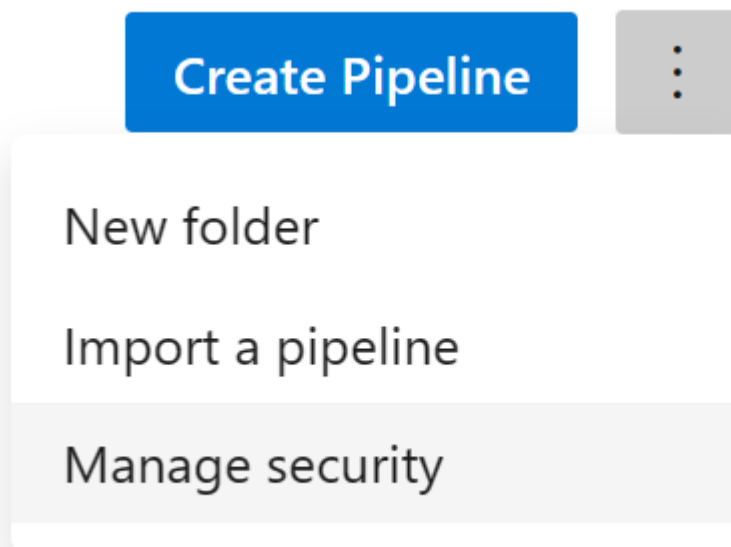
Azure DevOps may not import the three MLOps V2 repos with the default branch set to `main`. If not, select **Branches** under the **Repos** section on the left and [reset the default branch](#) to `main` for each of the three imported repos.

For an existing import, synchronize the latest personal `main` branch first. Selecting `main` in a saved Azure DevOps pipeline does not fetch later GitHub changes automatically.

4. Lastly, you will grant the MLOps Solution Accelerator permission to create new pipelines in the ML projects you will create. In your `mlops-v2` project, select the Pipelines section on the left side.



Select the three vertical dots next to **Create Pipeline** and select **Manage Security**.



Select the "<projectname> Build Service" account for your project under the Users section. Change the permissions for **Edit build pipeline** and **Create build pipeline** to **Allow**

Permissions for mlops-v2



Search for users or groups		mlops-v2 Build Service (REDACTED)	
▼ Azure DevOps Groups		Administer build permissions	Not set ▼
BA Build Administrators		Delete build pipeline	Not set ▼
C Contributors		Delete builds	Not set ▼
PA Project Administrators		Destroy builds	Not set ▼
R Readers		Edit build pipeline	Allow ▼ ✓
PA Project Collection Administrators		Edit build quality	Allow ▼
PA Project Collection Build Administrators		Manage build qualities	Not set ▼
PA Project Collection Build Service Accounts		Manage build queue	Allow ▼
PA Project Collection Test Service Accounts		Override check-in validation by build	Allow ▼
▼ Users		Queue builds	Not set ▼
MS mlops-v2 Build Service (REDACTED) 🗑️		Retain indefinitely	Not set ▼
		Stop builds	Not set ▼
		Update build information	Allow ▼
		View build pipeline	Allow ▼
		View builds	Allow ▼

You are done cloning and configuring the MLOps V2 Solution Accelerator. Next, you will create a new ML project using the accelerator templates.

Create and Configure a New ML Project Repo

In this section, you will create your ML project repository, set permissions to allow the solution accelerator to interact with your project, and create service principals so your Azure pipelines can interact with Azure Machine Learning.

Creating the project repository

1. Open the **Repos** drop-down once more and select **New repository**. Create the empty **target** repository for your ML project; do not import a finished project into it. The initializer will populate its files from `taxi-fare-regression-template`. The existing walkthrough and screenshots call this target `taxi-fare-regression`; use `taxi-fare-regression-demo` or another new name if that name is already your working project.

Leave **Add a README** selected to initialize the repo with a `main` branch.

Use a new, empty application repository. If your working project is already named `taxi-fare-regression`, choose another name such as `taxi-fare-regression-demo` throughout the remaining steps. Never initialize over the working application or the `taxi-fare-regression-template` source repository.

Create a repository



Repository type

 Git 

Repository name *

taxi-fare-regression



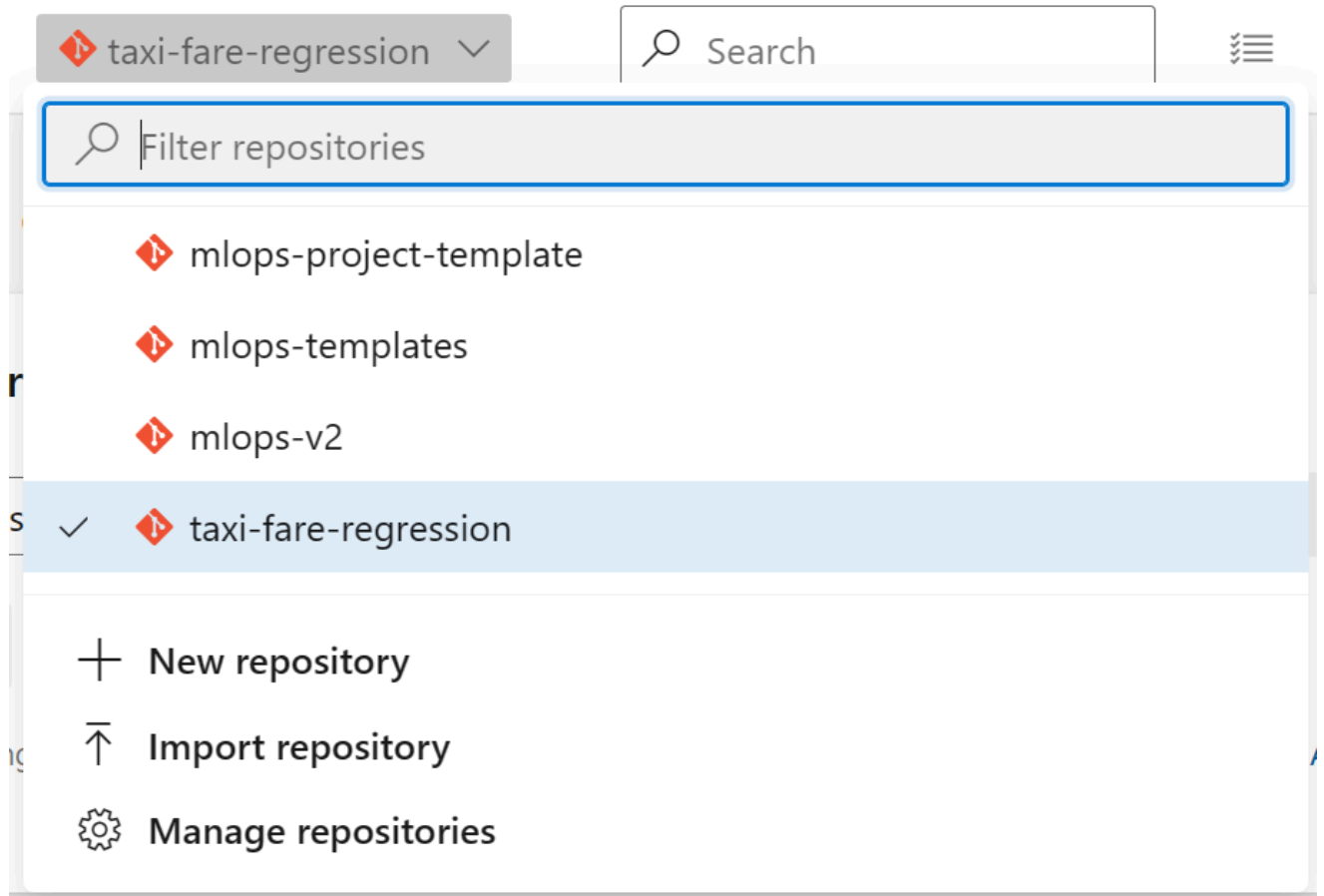
Add a README

Add a .gitignore: None



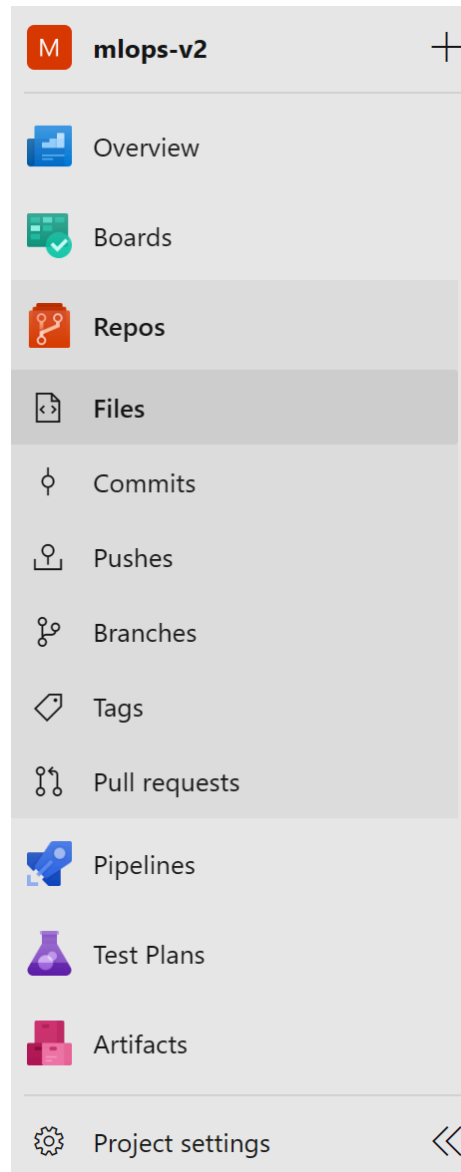
Your repository will be initialized with a  `main` branch.

You should now have your `taxi-fare-regression` repo and [all three MLOps V2 repositories](#) in your Azure DevOps project.



Setting project permissions

2. Next, set access permissions on your ML project repository. Open the **Project settings** at the bottom of the left hand navigation pane



Under the **Repos** section, select **Repositories**.

- Select the `taxi-fare-regression` repository.
- Select the **Security** tab.
- Under **User permissions**, select the "<projectname> Build Service" account for your project under the Users section.
- Change the permissions for **Contribute** and **Create branch** to **Allow**.

← All Repositories +

Filter by keywords

- mlops-project-template
- mlops-templates
- mlops-v2
- taxi-fare-regression**

Settings Policies **Security**

User permissions Download detailed report

☒ Inheritance ⓘ

Search for users or groups

Azure DevOps Groups

- BA Build Administrators
- C Contributors
- PA Project Administrators
- R Readers
- PA Project Collection Administrators
- PA Project Collection Build Service Accol
- PA Project Collection Service Accounts

Users

- MS mlops-v2 Build Service [redacted]

mlops-v2 Build Service [redacted]

Bypass policies when completing pull requests	Not set	▼
Bypass policies when pushing	Not set	▼
Contribute	Allow	▼ ✓
Contribute to pull requests	Not set	▼
Create branch	Allow	▼ ✓
Create tag	Allow (inherit...)	▼ ⓘ
Delete repository	Not set	▼
Edit policies	Not set	▼
Force push (rewrite history, delete branches and tags)	Not set	▼
Manage notes	Not set	▼
Manage permissions	Not set	▼
Read	Allow (inherit...)	▼ ⓘ

Initializing the new ML project repo

In this step, run the simplified `initialise-project` pipeline. It asks only for your Azure DevOps project, new target repository, and imported taxi source repository. It always creates the repaired classical / AML CLI v2 / Bicep project; unused scenario and upstream-template options are not shown.

For the repaired taxi selection, the YAML checks out the imported source and your already-created target, copies the working source files, commits and pushes the generated project, and registers its pipelines. It does not create personal GitHub forks, create the target repository object, or deploy Azure resources during initialization.

3. Open the Pipelines section again and select **Create Pipeline** in the center of the page.

Azure DevOps [redacted] / mlops-v2 / Pipelines

mlops-v2 +

- Overview
- Boards
- Repos
- Pipelines**
- Pipelines
- Environments
- Releases
- Library
- Task groups
- Deployment groups

Connect Select Configure Review

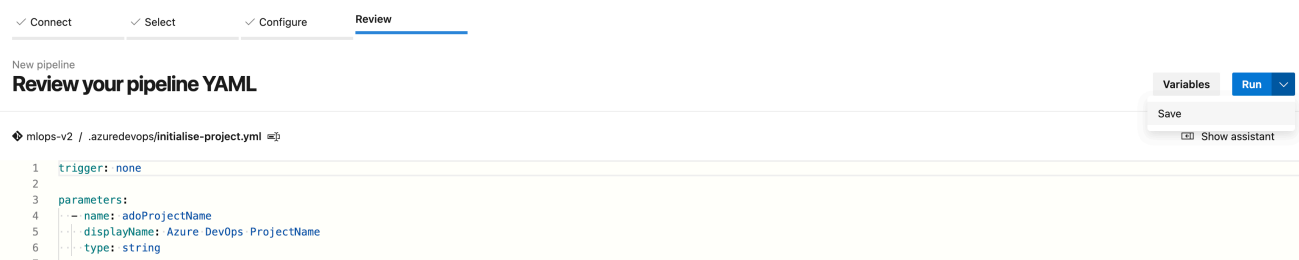
New pipeline

Where is your code?

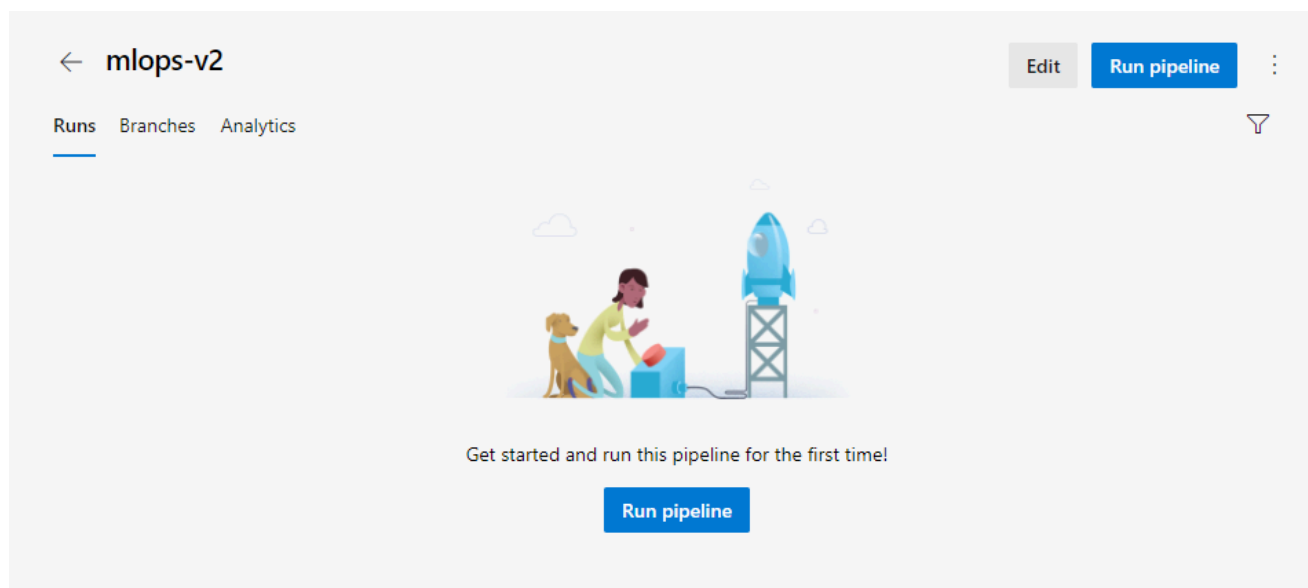
- Azure Repos Git** (YAML) Free private Git repositories, pull requests, and code search
- Bitbucket Cloud** (YAML) Hosted by Atlassian
- GitHub** (YAML) Home to the world's largest community of developers
- GitHub Enterprise Server** (YAML) The self-hosted version of GitHub Enterprise
- Other Git** Any generic Git repository
- Subversion** Centralized version control by Apache

- Select **Azure Repos Git**
- Select the **mlops-v2** repository imported from **alvinea28/own-ml-ops-v2** (or **own-ml-ops-v2** if you kept that Azure Repos name)
- Select **Existing Azure Pipelines YAML file**
- Ensure the selected branch is **main**
- Select the `/.azuredevops/initialise-project.yml` file in the Path drop-down
- Click Continue

On the pipeline review page, drop-down the **Run** menu and select **Save** the pipeline before running it.



Now select **Run pipeline**.



The **Run pipeline** dialog now has only the three project inputs below. Keep the built-in **Pipeline version** selector on **main** to use the current accelerator code. It is not an Azure environment selector.

Project Parameters

Input	What it means	Value to enter
Azure DevOps project containing the repositories (<code>adoProjectName</code>)	The Azure DevOps project, not your organization or GitHub name	Your actual project name, such as <code>fixed-mlops-v2</code> (or <code>mlops-v2</code> if following this walkthrough's example)
New taxi project repository (must already exist) (<code>repoName</code>)	The new empty target that receives the generated files	<code>taxi-fare-regression-demo</code> , or your chosen new target name

Input	What it means	Value to enter
Source template repository (already imported) (<code>taxiTemplateRepoName</code>)	The Azure Repos source holding the repaired code	<code>taxi-fare-regression-template</code>

Classical ML + AML CLI v2 + Bicep are automatic. The repaired-template checkbox, unused upstream-template field, and ML type/interface/provider selectors have been removed from this form. The required shared helper repo `mlops-templates` still needs to be imported; it is a dependency of the generated pipelines, not another input on this initialization form.

If the old fields are still visible: synchronize the latest personal `main` into Azure Repos, verify the saved pipeline's YAML path is `/.azuredevops/initialise-project.yml`, then close and reopen **Run pipeline**. Selecting `main` alone does not fetch GitHub changes into an existing Azure Repos import.

What did "other scenarios" mean? Computer vision, NLP, Python SDK, Responsible AI, Terraform, or the original upstream templates. If you need those, create a separate pipeline from `initialise-project-advanced.yml` and import `mlops-project-template`. It is not needed for the repaired taxi project, and the advanced path does not include the standalone taxi runtime fixes.

After selecting the parameters, click **Run** at the bottom of the panel. The first run of the pipeline will prompt you to grant access to the repositories you created.

 This pipeline needs permission to access 2 resources before this run can continue to Initialise Project
 View

Jobs

Permission needed

Name	Status	Duration
 Checkout repos, initialise your new one and create pipelines	Waiting	

Sources

Repository	Branch / tag	Version	Related
mlops-v2 Azure Repos	 main	663ca9cf	None
mlops-project-template Azure Repos	 main	2915b1ed	
taxi-fare-regression Azure Repos	 main	11c0acf5	

Click **View** to see the permissions waiting for review.

Waiting for review



Initialise Project

	Permission Permission needed	taxi-fare-regression Repository	Permit
	Permission Permission needed	mlops-project-tem... Repository	Permit

For each of the repos, click **Permit** waiting for review.

The pipeline run should take a few minutes. When the pipeline run is complete and successful, go back to **Repos** and look at the contents of your ML project repo, `taxi-fare-regression`. The solution accelerator has populated the project repository according to your configuration selections.

mlops-v2 / Repos / Files / taxi-fare-regression

Search

taxi-fare-regression

- > data
- > data-science
- > infrastructure
- > mlops
- config-infra-dev.yml
- config-infra-prod.yml

main

Type to find a file or folder...

Files

Contents History

Name ↑	Last change	Commits
data	35m ago	78065277 initial commit Azure Pipeline
data-science	35m ago	78065277 initial commit Azure Pipeline
infrastructure	35m ago	78065277 initial commit Azure Pipeline
mlops	35m ago	78065277 initial commit Azure Pipeline
config-infra-dev.yml	35m ago	78065277 initial commit Azure Pipeline
config-infra-prod.yml	35m ago	78065277 initial commit Azure Pipeline

Set up build Clone

The structure of the project repo is as follows:

File	Purpose
/data	Sample data for the example project
/data-science	Contains python code for the data science workflow
/infrastructure	IaC code for deploying the Azure Machine Learning infrastructure

File	Purpose
/mlops	Azure DevOps pipelines and Azure Machine Learning pipelines for orchestrating deployment of infrastructure and ML workflows.
config-infra-dev.yml	Configuration file to define dev environment resources
config-infra-prod.yml	Configuration file to define production environment resources

Create and Configure Service Principals and Connections

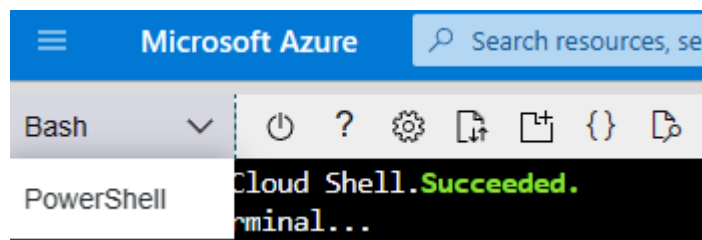
For Azure DevOps pipelines to create Azure Machine Learning infrastructure and deploy and execute Azure ML pipelines, it is necessary to create an Azure service principal for each Azure ML environment (Dev and/or Prod) and configure Azure DevOps service connections using those service principals.

Security Best Practice: Azure DevOps now supports **workload identity federation** as a more secure alternative to service principal secrets. Workload identity federation uses OpenID Connect (OIDC) to establish trust without storing long-lived secrets. For production deployments, consider using workload identity federation instead of the service principal methods described below. See [Microsoft's documentation](#) for setup instructions.

These service principals can be created using one of the two methods below:

Create Service Principal from Azure Cloud Shell

1. Launch the [Azure Cloud Shell](#) . (If this the first time you have launched the cloud shell, you will be required to create a storage account for the cloud shell.)
2. If prompted, choose **Bash** as the environment used in the Cloud Shell. You can also change environments in the drop-down on the top navigation bar



3. Copy the bash commands below to your computer and update the **projectName**, **subscriptionId**, and **environment** variables with the values for your project. If you are creating both a Dev and Prod environment you will need to run this script once for each environment, creating a service principal for each. This command will also grant the **Contributor** role to the service principal in the subscription provided. This is required for Azure DevOps to properly deploy resources to that subscription.

```
projectName="<your project name>"
roleName="Contributor"
subscriptionId="<subscription Id>"
environment="<Dev|Prod>" #First letter should be capitalized
servicePrincipalName="Azure-ARM-${environment}-${projectName}"
```

```
# Verify the ID of the active subscription
echo "Using subscription ID $subscriptionId"
echo "Creating SP for RBAC with name $servicePrincipalName, with role $roleName and in scopes
/subscriptions/$subscriptionId"
az ad sp create-for-rbac --name $servicePrincipalName --role $roleName --scopes
/subscriptions/$subscriptionId
echo "Please ensure that the information created here is properly save for future use."
```

4. Copy your edited commmands into the Azure Shell and run them (**Ctrl** + **Shift** + **v**).

```
kevin@Azure:~$ projectName="Sample_Project_Name"
roleName="Contributor"
subscriptionId="fbf19cb9-0e29-493b-abad-82713041d5a3"
environment="Dev" #First letter should be capitalized
servicePrincipalName="Azure-ARM-${environment}-${projectName}"
# Verify the ID of the active subscription
echo "Using subscription ID $subscriptionId"
echo "Creating SP for RBAC with name $servicePrincipalName, with role $roleName and in scopes /subscriptions/$subscriptionId"
az ad sp create-for-rbac --name $servicePrincipalName --role $roleName --scopes /subscriptions/$subscriptionId
echo "Please ensure that the information created here is properly save for future use."
Using subscription ID
Creating SP for RBAC with name Azure-ARM-Dev-Sample_Project_Name, with role Contributor and in scopes /subscriptions/fbf19cb9-0e29-493b-abad-82713041d5a3
Creating 'Contributor' role assignment under scope '/subscriptions/fbf19cb9-0e29-493b-abad-82713041d5a3'
The output includes credentials that you must protect. Be sure that you do not include these credentials in your code or check the credentials into your source control.
For more information, see https://aka.ms/azadsp-cli
{
  "appId": "074da5fb-8ffc-42cd-af0f-b48f1d1cb9e8",
  "displayName": "Azure-ARM-Dev-Sample_Project_Name",
  "password": "6QK8Q~CKo3x1jQxyrTQA3pC3T1KImIzrde9cLaau",
  "tenant": "20a1d97b-1261-47c9-8cec-adba8291275f"
}
Please ensure that the information created here is properly save for future use.
kevin@Azure:~$
```

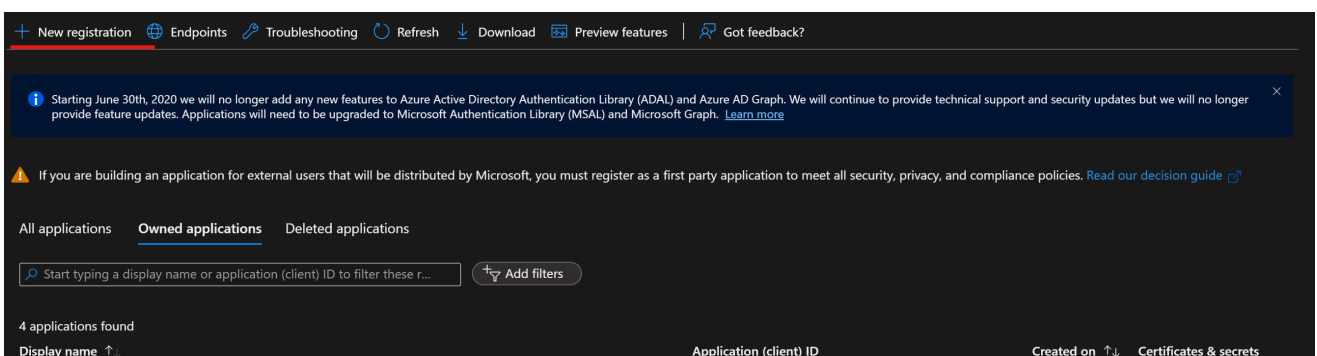
5. After running these commands you will be presented with information related to the service principal. Save this information to a safe location, it will be used later in the demo to configure Azure DevOps.

```
{
  "appId": "<application id>",
  "displayName": "Azure-ARM-dev-Sample_Project_Name",
  "password": "<password>",
  "tenant": "<tenant id>"
}
```

6. Repeat **step 3** if you are creating service principals for Dev and Prod environments.
7. Close the Cloud Shell once the service principals are created.

Create Service Principal from the Azure Portal

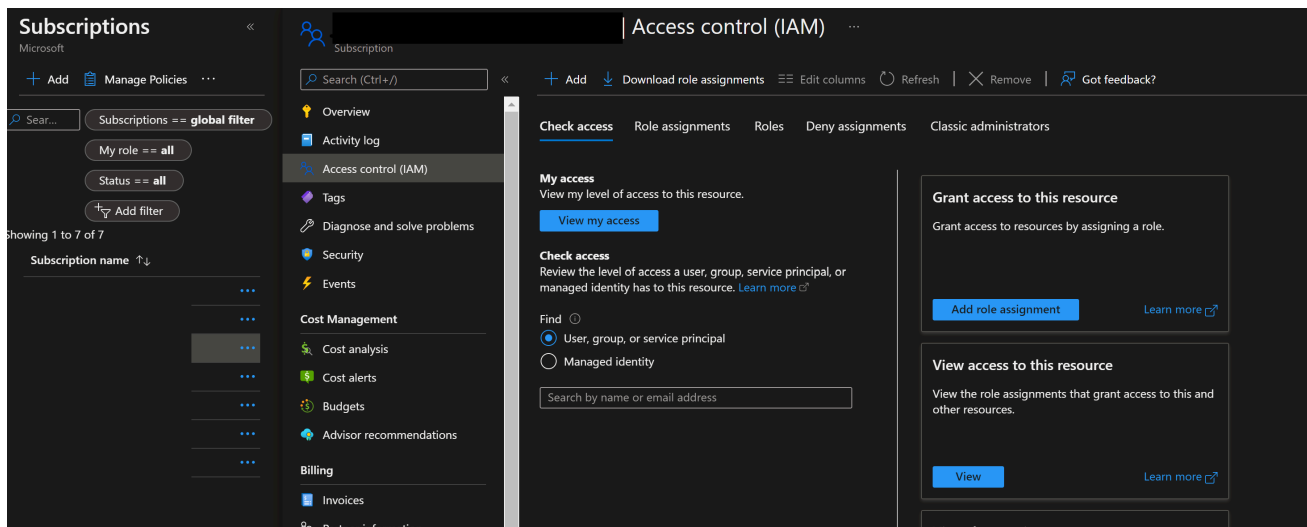
1. Navigate to [Azure App Registrations](#)
2. Select **New registration**.



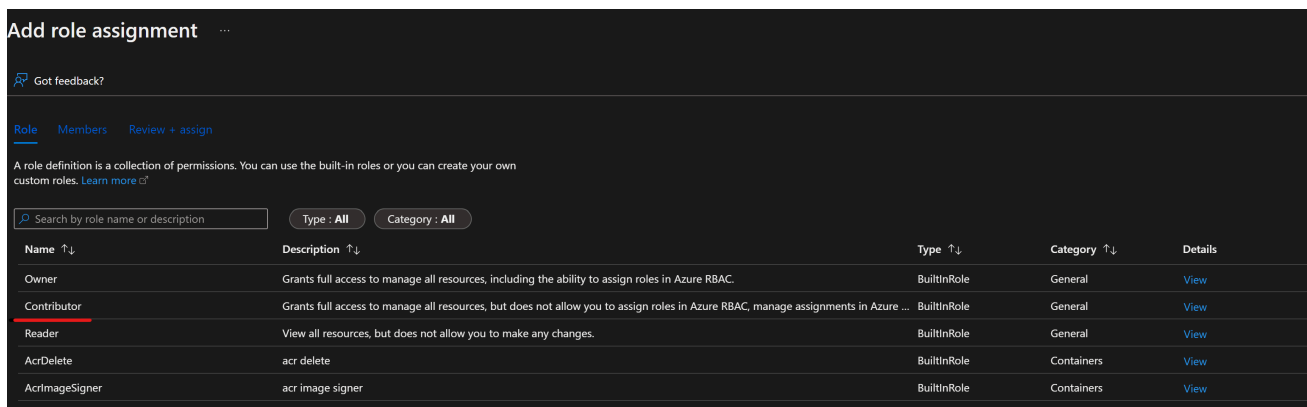
3. Go through the process of creating a Service Principle (SP) selecting "Accounts in any organizational directory (Any Azure AD directory - Multitenant)" and name it "Azure-ARM-Dev-ProjectName". Once created, repeat and create a new SP named "Azure-ARM-Prod-ProjectName". Please replace "ProjectName" with the name of your

project so that the service principal can be uniquely identified.

- Go to "Certificates & Secrets" and add for each SP "New client secret", then store the value and secret separately.
- To assign the necessary permissions to these principals, select your respective [subscription](#) and go to **Access control (IAM)**. Select +Add then select "Add Role Assignment".



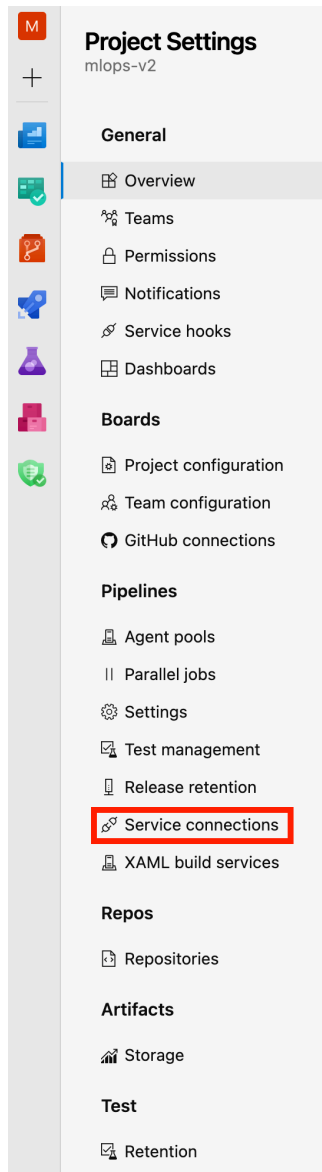
- Select Contributor and add members selecting + Select Members. Add the member "Azure-ARM-Dev-ProjectName" as create before.



- Repeat step here, if you deploy Dev and Prod into the same subscription, otherwise change to the prod subscription and repeat with "Azure-ARM-Prod-ProjectName". The basic SP setup is successfully finished.

Create Service Connections

Select **Project Settings** at the bottom left of the project page and select **Service connections**.



Select **Create service connection**

- For service, select **Azure Resource Manager** and **Next**
- For identity type, select **App registration (automatic)**
- For Credential, select **Workload identity Federation**
- for Subscription, select your subscription from the drop-down and select **Save**

New Azure service connection

Azure Resource Manager

Identity type

App registration (automatic)
Recommended

[Need help choosing a connection type?](#)

Credential

Workload identity federation
Recommended

Scope level

☒ Subscription
☐ Management Group
☐ Machine Learning Workspace

Subscription

If you can't find subscription you need, please [create manually](#).

Resource group

Specify to limit access to the chosen resource group only.

Service Connection Name

Azure-ARM-Prod

Service Management Reference (optional)

Provide a Service Management Reference if your organization requires one to be configured on Entra ID App registrations.

Description (optional)

Security

☐ Grant access permission to all pipelines

[Learn more](#)
[Troubleshoot](#)

Back
Save

Complete the new service connection configuration using the information from your tenant, subscription, and the service principal you created for Prod.

Name this service connection **Azure-ARM-Prod**. Check **Grant access permission to all pipelines**, and click **Verify and save**.

Create Azure DevOps Environment

The pipelines in each branch of your ML project repository will depend on an Azure DevOps environment. These environments should be created before deployment.

To create the prod environment, select **Pipeline** in the left menu and **Environments**. Select **New environment**

New environment



Name ⓘ

prod

Description ⓘ

Describe the environment

Resource



None

You can add resources later



Kubernetes

Add Kubernetes namespace



Virtual machines

Manage virtual machines

Name the new environment `prod` and click **Create**. The environment will initially be empty and indicate "Never deployed" but this status will update after the first deployment.

The configuration of your new ML project repo is complete and you are ready to deploy your Azure Machine Learning infrastructure and deploy ML training and model deployment pipelines in the next section.

Deploy and Execute Azure Machine Learning Pipelines

Now that your ML project is created, this last section will guide you through executing Azure DevOps pipelines created for you that will first deploy Azure Machine Learning infrastructure for your project then deploy a model training pipeline followed by a model deployment pipeline.

Each pipeline may have different roles associated with its deployment and management. For example, infrastructure by your IT team, model training by your data scientists and ML engineers, and model deployment by ML engineers. Likewise, depending on the environments and project branches you have created, you may deploy infrastructure for both **dev** and **prod** Azure ML infrastructure, with data scientists developing the training pipeline in the **dev** environment and branch and, when the model is acceptable, opening a pull request to the **main** branch to merge updates and run the model deployment pipeline in the **prod** environment.

Depending on the options you chose when initializing the project, you should have one infrastructure deployment pipeline, one model training pipeline, and one or two model deployment pipelines in your ML project. Model deployment options are online-endpoint for near real-time scoring and batch-endpoint for batch scoring. To see all pipelines in your project, select the **Pipelines** section from the left navigation menu, then **Pipelines**, then the **All** tab. For the example project in this guide, you should see:

Pipelines

New pipeline

Recent **All** Runs

Filter pipelines

All pipelines

New folder

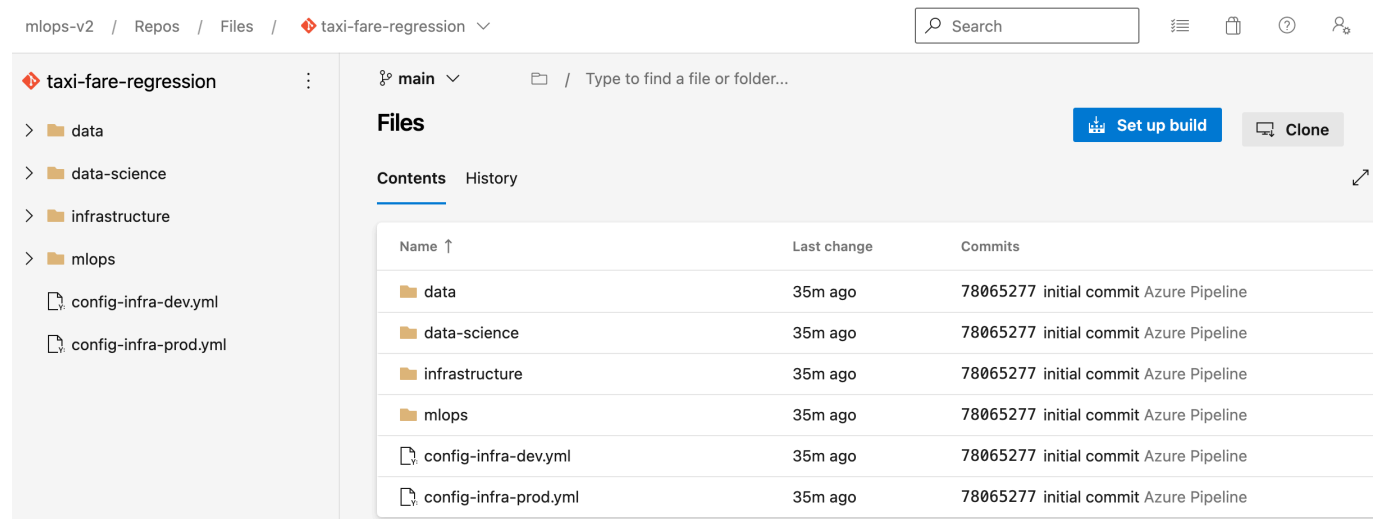
Name

- taxi-fare-regression
 - infrastructure
 - bicep-ado-deploy-infra No runs yet
 - mlops
 - deploy-batch-endpoint-pipeline No runs yet
 - deploy-model-training-pipeline No runs yet
 - deploy-online-endpoint-pipeline No runs yet
- mlops-v2 #20230331.1 • adding context/clarification for when using WSL (#89)

Deploy Azure Machine Learning Infrastructure

The first task for your ML project is to deploy Azure Machine Learning infrastructure in which to develop your ML code, define your datasets, define your ML pipelines, train models, and deploy your models in production. This pipeline deployment is typically managed by your IT group responsible for ensuring that the subscription is able to create the infrastructure needed. The infrastructure is created by executing the Azure DevOps deploy-infra pipeline. Before doing this, you will customize environment files that define unique Azure resource groups and Azure ML workspaces for your project.

To do this, go back to **Repos** and your ML project repo, in this example, `taxi-fare-regression`. You will see two files in the root directory, `config-infra-prod.yml` and `config-infra-dev.yml`.



ESv3 compute sizes and quota budget

The repaired taxi template uses **Standard ESv3 Family Cluster Dedicated vCPUs**, not the older DSv3/DSv2 examples. The environment files default to **eastus**; check quota in the exact subscription and region used by your service connection. Select the **Azure Machine Learning** quota provider, not the ordinary VM quota.

Component	VM size	vCPUs / RAM per node	Scaling / tier	Quota budget
Bicep <code>cpu-cluster</code> , reused by training	Standard_E4s_v3	4 / 32 GiB	Dedicated , 0–4 nodes	16 cores
Batch <code>batch-cluster</code>	Standard_E4s_v3	4 / 32 GiB	Dedicated , 0–5 nodes	20 cores
Managed online deployment	Standard_E2s_v3	2 / 16 GiB	1 instance	4 cores , including upgrade reserve
Total per environment				40 cores

Bicep and training configure the **same** training cluster, so do not count it twice. Batch's job-level `instance_count: 2` is separate from the five-node cluster maximum. Dedicated and low-priority quota are different; this profile does not use low-priority compute.

The supplied screenshot reported **96 available ESv3 Dedicated cores**. That is a planning snapshot, not an independently verified live quota or a guarantee of regional VM capacity. One environment's 40-core maximum fits that budget; dev and prod together would need **80 cores** when they share a subscription and region. Allow for other workspaces, total regional Dedicated-core limits, workspace caps, and overlapping deployments. Online E2s_v3 uses $\text{ceil}(1.2 \times \text{instance_count}) \times 2$ quota cores, so one instance requires four.

Set `aml_compute_sku: Standard_E4s_v3` in `config-infra-dev.yml` and `config-infra-prod.yml`. The Bicep validation/deployment pipeline and training/batch pipelines consume the shared configuration. Scaling variables are `aml_compute_min_instances: 0`, `aml_compute_max_instances: 4`, and `aml_batch_max_instances: 5`. The standalone manual batch definition and online definition remain in `mlops/azureml/deploy/batch/batch-cluster.yml` and `mlops/azureml/deploy/online/online-deployment.yml`; keep them aligned if overriding the defaults. This profile describes the repaired **Bicep** path, not the separate upstream Terraform/CV/NLP templates.

Existing cluster warning: the shared compute helper skips a cluster that already exists. Changing these settings does not resize an old cluster or change its tier. Inspect the current resources first and plan any replacement explicitly; do not delete an active cluster or assume immutable properties can be updated. Keep minimum nodes at zero to allow idle training/batch scale-down. A single online instance is not high availability and remains allocated while deployed.

See the template's [compute profile guide](#), [supported online SKUs](#), and [Azure ML quota/reservation rules](#).

Making sure you are in the **main** branch, click on `config-infra-prod.yml` to open it.

Under the Global section, you will see properties for `namespace`, `postfix`, and `location`.

```
# Prod environment
variables:
  # Global
  ap_vm_image: ubuntu-22.04

  namespace: mlopsv2 #Note: A namespace with many characters will cause storage account creation to fail due
to storage account names having a limit of 24 characters.
  postfix: 0001
  location: eastus
  environment: prod
  enable_aml_computecluster: true
  aml_compute_sku: Standard_E4s_v3
  aml_compute_min_instances: 0
  aml_compute_max_instances: 4
  aml_batch_max_instances: 5
```

The two properties `namespace` and `postfix` will be used to construct a unique name for your Azure resource group, Azure ML workspace, and associated resources. The naming convention for your resource group will be `rg-<namespace>-<postfix>prod`. The name of the Azure ML workspace will be `mlw-<namespace>-<postfix>prod`. The `location` property will be the Azure region into which to provision these resources.

Edit `config-infra-prod.yml` to set the variables for your environment. You can clone the repo, edit the file, and push/PR to make the change or select **Edit** in the upper right of the screen to edit the file within Azure DevOps. If editing in place, change `namespace`, `postfix`, and `location` to your preferences and click **Commit**

If the `enable_aml_computecluster` property is set to true, the infra deployment pipeline will pre-create Azure ML compute clusters for your training. In the case of CV or NLP scenarios, it will create both CPU-based and GPU-based compute clusters so ensure that your subscription has GPU compute available.

Now you are ready to run the infrastructure deployment pipeline. Open the **Pipelines** section again and select **New pipeline** in the upper right of the page.

 **mlops-v2**

 Overview
  Boards
  Repos
  **Pipelines**
 Environments
  Releases
  Library
  Task groups
  Deployment groups

Connect Select Configure Review

New pipeline
Where is your code?

 **Azure Repos Git** YAML
 Free private Git repositories, pull requests, and code search

 **Bitbucket Cloud** YAML
 Hosted by Atlassian

 **GitHub** YAML
 Home to the world's largest community of developers

 **GitHub Enterprise Server** YAML
 The self-hosted version of GitHub Enterprise

 **Other Git**
 Any generic Git repository

 **Subversion**
 Centralized version control by Apache

- Select **Azure Repos Git**
- Select the **taxi-fare-regression** repository
- Select **Existing Azure Pipelines YAML file**
- Ensure the selected branch is **main**
- Select the `/infrastructure/pipelines/bicep-ado-deploy-infra.yaml` file in the Path drop-down
- Click Continue

Now you will see the pipeline details.

✓ Connect ✓ Select ✓ Configure **Review**

New pipeline
Review your pipeline YAML

Variables Run

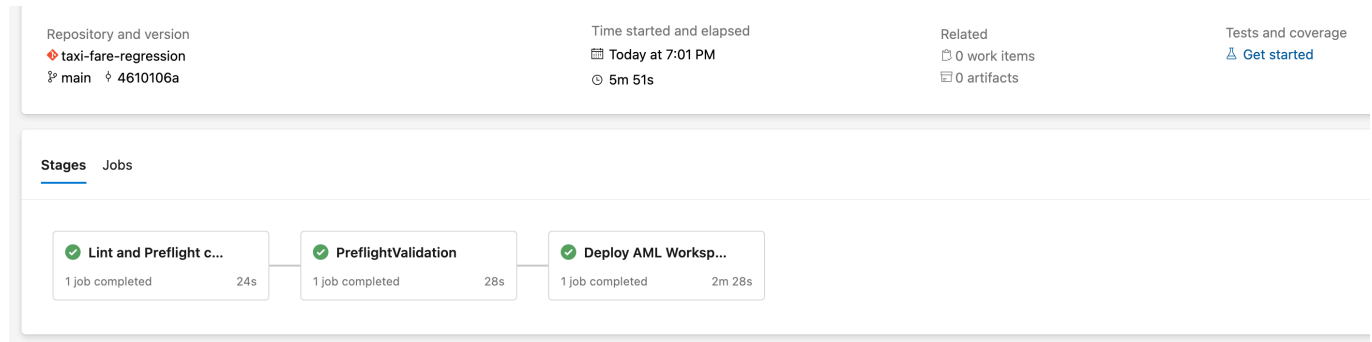
taxi-fare-regression / infrastructure/pipelines/bicep-ado-deploy-infra.yaml

```

1  Copyright (c) Microsoft Corporation. All rights reserved.
2  # Licensed under the MIT License.
3
4  variables:
5  - ${{ if eq(variables['Build.SourceBranchName'], 'main') }}:
6  |   # 'main' branch: PRD environment
7  |   template: ../../config-infra-prod.yaml
8  - ${{ if ne(variables['Build.SourceBranchName'], 'main') }}:
9  |   # 'develop' or feature branches: DEV environment
10 |   template: ../../config-infra-dev.yaml
11

```

Click **Run** to execute the pipeline. This will take a few minutes to finish. When complete, you can view the pipeline jobs and tasks by selecting **Pipelines** then **taxi-fare-regression** under **Recently run pipelines**.



The pipeline should create the following artifacts which you can view in your Azure subscription:

- Resource Group for your Workspace
- Azure Machine Learning Workspace and associated resources including Storage Account, Container Registry, Application Insights, and Keyvault
- Inside the workspace, an AmlCompute cluster will be created

After the full infrastructure deployment succeeds, complete the identity and registry-access checks below before running training. A lint or preflight-only run does not create the workspace, compute identity, or registry role assignments.

Before training: managed identities and container registry access

The workspace identity and the training compute identity are different. Do not grant a role to the workspace and assume the `cpu-cluster` automatically uses that principal. Likewise, the `Azure-ARM-Prod` service connection identity is the deployment caller, not the identity used by compute nodes to pull images.

Identity	What to verify	Why it matters
Azure ML workspace system-assigned identity	Workspace → Identity → System assigned is On ; record its principal/object ID	The workspace uses this identity to access associated services. The workspace Bicep already declares <code>SystemAssigned</code> .
<code>cpu-cluster</code> managed identity	Inspect the actual training cluster's identity and its own principal/object ID	This is the identity that needs permission to pull the training image. The current compute Bicep does not explicitly declare an identity or ACR role assignment.
Azure DevOps service connection	The pipeline is authorized to use the intended connection	Deployment/validation permissions do not automatically grant image-pull access to the compute identity.

1. Verify the workspace and compute identities

1. In the Azure portal, open the deployed **Azure Machine Learning workspace** → **Identity** → **System assigned**. Verify it is **On**. Bicep should already enable it when the workspace is deployed; a system-assigned identity is created with its resource, not as a separate user-assigned identity resource.
2. In Azure ML Studio, open **Compute** → **Compute clusters** → **cpu-cluster** and inspect its managed-identity settings. Enable a **system-assigned identity** for the cluster if it has none, using the supported portal/Studio controls or [the documented compute-identity update procedure](#). Do not delete or recreate a running cluster merely to add identity access.
3. Record the **compute principal/object ID** separately from the workspace ID. If the cluster intentionally uses a user-assigned identity, grant access to the identity actually selected by that compute instead of replacing it blindly.

4. Verify the actual Azure resource after the change. The shared creation helper skips a cluster that already exists; rerunning it does not repair a missing identity. A system-assigned workspace identity cannot be attached to the cluster as though it were a user-assigned identity.

2. Select the correct registry role

Open the **container registry associated with this workspace**, then **Settings** → **Properties** → **Role assignment permissions mode**. Choose the role for that mode:

Registry mode or requirement	Role for the compute identity	Notes
RBAC Registry Permissions	AcrPull	Grants image-pull access. Scope the assignment to this registry, not the entire subscription.
RBAC Registry + ABAC Repository Permissions	Container Registry Repository Reader	Use the ABAC-compatible data role; legacy <code>AcrPull</code> is not honored in this mode. Add Container Registry Repository Catalog Lister only when the workflow must list repositories, or when reproducing the catalog-list access of <code>AcrPull</code> .
A documented need to read registry configuration/control-plane metadata	Reader , or the narrower Container Registry Configuration Reader and Data Access Configuration Reader	Additional read visibility is not a blanket requirement for image pulls. In ABAC mode, generic <code>Reader</code> does not grant image-pull data access.

Do not treat `Reader` and `Container Registry Repository Reader` as the same role. There is no universal requirement to assign both `AcrPull` and a repository reader role: use the role appropriate to the registry mode and the operation. Do not switch a live registry's permission mode just to follow this guide; assess Azure ML image-build compatibility and existing role assignments before any migration.

3. Assign access on the container registry

1. Open **Container Registry** → **Access control (IAM)** → **Add** → **Add role assignment**.
2. Select **AcrPull** for the RBAC-only path above, or the appropriate ABAC role if that is the registry's existing mode. Select **Next**.
3. Under **Members**, choose **Managed identity**, then select the training compute identity. Verify its principal/object ID matches `cpu-cluster` from step 1, not just a similar display name or the workspace identity.
4. Keep scope at the registry resource. For ABAC conditions, ensure the permitted repositories cover the training image paths. Use **Review + assign**.
5. If control-plane reader access is specifically required, add the selected reader role separately at the same registry scope. Do not grant Owner, Contributor, or subscription-wide access merely to fix an image pull.
6. Confirm the assignments in **IAM** → **Role assignments** and allow time for RBAC propagation before starting the training run. Some supported Azure ML creation flows assign `AcrPull` automatically; verify rather than adding duplicates.

The person assigning roles needs permission to create role assignments at that scope, such as **Role Based Access Control Administrator**, **User Access Administrator**, or **Owner**. Contributor alone does not normally grant this. The ACR admin account can remain disabled; do not enable shared admin credentials as a substitute for managed-identity access.

4. Final checks before clicking Run on training

- The workspace and `cpu-cluster` exist, and the compute has the expected identity.
- The **compute** principal has the correct image-pull role on the correct ACR. Assigning a role only to the workspace principal does not authorize the compute.
- Preserve the workspace's required dependency/image-build permissions. If the workspace or a build identity must build/push images, read/pull-only roles are not sufficient; diagnose that identity separately instead of broadening the training compute's permissions.
- Registry network/firewall/private-endpoint access also permits the compute; an IAM role does not bypass network controls. Wait for role propagation.
- Data-storage permissions are a separate requirement. Check them if training uses identity-based datastores; successful image pull does not prove data access.
- A successful infrastructure preflight is not an image-pull test. Start a new training run only after these checks, then inspect its environment setup logs.

Repeat the compute-identity checks for `batch-cluster` before batch inference. Managed online endpoints use their own endpoint identity; do not assume workspace or training-cluster role assignments automatically apply to online serving.

References: [Azure ML service authentication and compute image pulls](#), [ACR roles](#), and [ABAC repository permissions](#).

Deploy Azure Machine Learning Model Training Pipeline

The solution accelerator includes code and data for a sample end-to-end machine learning pipeline which trains a linear regression model to predict taxi fares in NYC. The pipeline is made up of multiple steps for data prep, training, model evaluation, and model registration. Sample pipelines and workflows for the Computer Vision and NLP scenarios will have different steps.

Complete [Before training: managed identities and container registry access](#) before starting this pipeline. The optional compute-instance examples below are not a replacement for configuring the actual `cpu-cluster` identity.

In this section you will execute an Azure DevOps pipeline that will create and run an Azure Machine Learning pipeline. Together, they perform the following steps:

- Connect to the Azure Machine Learning workspace created by the infrastructure deployment
- Reuse the verified `cpu-cluster` for training; the helper can create missing compute but does not replace the pre-training identity and registry-access checks
- Register the training dataset in the workspace
- Prepare data for training
- Registers a custom python environment with the packages required for this model
- Train a linear regression model to predict taxi fares
- Evaluate the model on the test dataset against the performance of any previously-registered models
- If the new model performs better, register the model as an MLflow model in the workspace for later deployment

Create Compute Instances with SystemAssigned/UserAssigned/No Managed Identity

In order to create a compute instance with or without managed identity, you can leverage the `/mlops-templates/templates/python-sdk-v2/create-compute-instance.yml` located within the **mlops-templates** repository.

These are optional development compute instances, not the `cpu-cluster` used by the taxi training pipeline. They are not required for this walkthrough. Each `Standard_E4s_v3` instance adds four ESv3 cores beyond the 40-core environment budget above. Use the same configured region and check additional available quota.

If you want to create a **compute instance without a managed identity** reference, you can add the following snippet with your own parameters to the `/mlops/devops-pipelines/deploy-model-training-pipeline.yml` pipeline definition:

```
-
  template: templates/python-sdk-v2/create-compute-instance.yml@mlops-templates
  parameters:
    instance_name: compute-instance-a
    size: Standard_E4s_v3
    location: ${ variables.location }
    description: compute instance a
```

In order to **create a system-assigned managed identity** and assign it your compute instance during creation, the above snippet can be adjusted as follows:

```
-
  template: templates/python-sdk-v2/create-compute-instance.yml@mlops-templates
  parameters:
    instance_name: compute-instance-a
    size: Standard_E4s_v3
    location: ${ variables.location }
    description: compute instance a
    identity_type: SystemAssigned
```

Lastly, to leverage a **user-assigned managed identity** for your compute, the following snippet can be used and adjusted as needed:

```
-
  template: templates/python-sdk-v2/create-compute-instance.yml@mlops-templates
  parameters:
    instance_name: compute-instance-a
    size: Standard_E4s_v3
    location: ${ variables.location }
    description: compute instance a
    identity_type: UserAssigned
    user_assigned_identity: e12c9326-0618-4036-a0a7-ad3bb396dc97
```

To deploy the model training pipeline, open the **Pipelines** section again and select **New pipeline** in the upper right of the page

- Select **Azure Repos Git**
- Select the **taxi-fare-regression** repository
- Select **Existing Azure Pipelines YAML file**
- Ensure the selected branch is **main**
- Select the `/mlops/devops-pipelines/deploy-model-training-pipeline.yml` file in the Path drop-down

- Click Continue

Next you can see the pipeline details.

Review your pipeline YAML

```

1 | Copyright (c) Microsoft Corporation. All rights reserved.
2 | # Licensed under the MIT License.
3 |
4 | name: deploy-model-training-pipeline
5 |
6 | > variables: --
15 |
16 |   trigger: none
17 |
18 | > pool: --
20 |
21 | > resources: --
27 |
28 |   stages:
29 |     - stage: DeployTrainingPipeline
30 |       displayName: Deploy Training Pipeline
31 |       jobs:
32 |         - job: DeployTrainingPipeline
  
```

Click **Run** to execute the pipeline. This will take several minutes to finish.

Important:

- It is noted that, with Free/Trial and other learning purpose subscriptions pipeline execution might take up to 90 minutes.
- In case, your model training pipeline execution in DevOps fails after running for a long period of time, but you have validated that a model is registered in Model Registry, it confirms that your model training is Azure ML was successful.
- This issue with longer running pipelines in Azure DevOps has been reported and is being tracked by product teams.

When complete, you can view the pipeline jobs and tasks by selecting **Pipelines** then **taxi-fare-regression** under **Recently run pipelines**. The pipeline run will be tagged `#deploy-model-training-pipeline`. Drill down into the pipeline run to see the **DeployTrainingPipeline** job. Click on the job to see pipeline run details.

Jobs			
Name	Status	Duration	
✓ DeployTrainingPipeline	Success	🕒 19m 9s	

Sources			
Repository	Branch / tag	Version	Related
taxi-fare-regression Azure Repos	📁 main	86571b9a	None
mlops-templates Azure Repos	📁 main	aa35e600	

Now you can open your Azure Machine Learning workspace to see the training run artifacts. Open a browser to <https://ml.azure.com> and login with your Azure account. You should see your Azure Machine Learning workspace under the **Workspaces** tab on the left. Your workspace name will have a name built from the options you chose in the `config-infra-prod.yml` file with the format **mlw-(namespace)-(postfix)(environment)**. For example, **mlw-mlopsv2-0001prod**. Click on your workspace. You will be presented with the Azure ML Studio home page for the workspace showing your training pipeline jobs under **Recent jobs**.

Recent jobs ...								View all
Display name	☆	Experiment	Status	Created on	Duration	Created by	Job type	
register_model	⋮	prod_taxi_fare_train_main	✓ Completed	Apr 5, 2023 2:08 PM	23s	Service Principal	Pipeline s...	
evaluate_model	⋮	prod_taxi_fare_train_main	✓ Completed	Apr 5, 2023 2:07 PM	21s	Service Principal	Pipeline s...	
train_model	⋮	prod_taxi_fare_train_main	✓ Completed	Apr 5, 2023 2:06 PM	43s	Service Principal	Pipeline s...	
prep_data	⋮	prod_taxi_fare_train_main	✓ Completed	Apr 5, 2023 1:53 PM	51s	Service Principal	Pipeline s...	
prod_taxi_fare_run_117	⋮	prod_taxi_fare_train_main	✓ Completed	Apr 5, 2023 1:53 PM	15m 14s	Service Principal	Pipeline	

You can now explore the artifacts of the pipeline run from the workspace navigation menus on the left:

- Select **Data** to see and explore the registered `taxi-data` Data asset.
- Select **Jobs** to see and explore the `prod_taxi_fare_train_main` Experiment.
- Select **Pipelines** to see and explore the `prod_taxi_fare_run` pipeline run.
- Select **Environments** then **Custom environments** to see the custom `taxi-train-env` environment registered by the pipeline.
- Select **Models** to see the `taxi-model1` registered by the training pipeline.
- Select **Compute** then **Compute clusters** to see the `cpu-cluster` created by the training pipeline.

This section demonstrated end-to-end model training in Azure DevOps/Azure ML pipelines, creating all necessary assets as part of the pipeline. With a trained model registered in the Azure Machine Learning workspace, the next section will guide you through deploying the model to as either a real-time endpoint or batch scoring endpoint.

Deploy Azure Machine Learning Model Deployment Pipeline

In this section you will execute an Azure DevOps pipeline that will create and run an Azure Machine Learning pipeline that deploys your trained model to an endpoint. This can be a online managed (real-time) endpoint called by an application to score new data or a batch managed endpoint to score larger blocks of new data. In this example, there are two sample model deployment pipelines provided, one for online endpoint and one for batch endpoint. You can deploy one or the other or both.

For each type of endpoint, the deployment steps are essentially the same:

- Connect to the Azure Machine Learning workspace created by the infrastructure deployment
- Create a new compute cluster (batch managed endpoint only)
- Create an endpoint in Azure Machine Learning for the model deployment
- Create a deployment of the trained model on the new endpoint
- Update traffic allocation for the endpoint
- Test the deployment with sample data

For batch managed endpoint deployments, a new compute cluster is created to process batch scoring requests. For online managed endpoints, the compute to process requests is managed by Azure Machine Learning.

To deploy the model deployment pipeline, open the **Pipelines** section again and select **New pipeline** in the upper right of the page

- Select **Azure Repos Git**
- Select the **taxi-fare-regression** repository

- Select **Existing Azure Pipelines YAML file**
- Ensure the selected branch is **main**
- Select `/mlops/devops-pipelines/deploy-online-endpoint-pipeline.yml` or `/mlops/devops-pipelines/deploy-batch-endpoint-pipeline.yml` in the Path drop-down depending on your choice
- Click Continue

Again, from the pipeline details path, click **Run** to execute the pipeline. This will take several minutes to finish. When complete, you can view the pipeline jobs and tasks by selecting **Pipelines** then **taxi-fare-regression** under **Recently run pipelines**. The pipeline run will be tagged `#deploy-online-endpoint-pipeline` or `#deploy-batch-endpoint-pipeline`. Drill down into the pipeline run to see the **DeployOnlineEndpoint** or **DeployBatchEndpoint** job and click on the job to see pipeline run details.

Once the deployment pipeline execution is complete, open your Azure Machine Learning workspace to see the deployed endpoints. Select **Endpoints** from the workspace navigation menu on the left. By default, you will see a list of deployed **Online endpoints**. If you chose to deploy the sample online endpoint pipeline, you should see your `taxi-online-(namespace)(postfix)prod` endpoint.

Endpoints

Real-time endpoints

Batch endpoints

+ Create Refresh Delete Edit columns Reset view

Showing 1-1 endpoints

Name	☆	Description	Created on	Created by	Updated on ↓	Compute type
taxi-online-sdtest0042prod		taxi cost online endpoint	Apr 5, 2023 7:16 PM	Service Principal	Apr 5, 2023 7:27 PM	Managed

Click on this endpoint instance to explore the details of the online endpoint model deployment.

If you deployed the batch managed endpoint, select **Batch endpoints** on the **Endpoints** page to see your `taxi-batch-(namespace)(postfix)prod` endpoint. Click on this endpoint instance to explore the details of the batch endpoint model deployment.

Endpoints

Real-time endpoints

Batch endpoints

+ Create Refresh Delete Edit columns Reset view

Showing 1-1 endpoints

Name	Description	Created on	Created by	Updated on	Tags
taxi-batch-sdtest0042prod	taxi cost batch endpoint	Apr 5, 2023 7:44 PM	Service Principal	Apr 5, 2023 7:44 PM	

For the batch endpoint, you can also select **Compute** and ***Compute clusters** to see the cluster created to support batch request processing.

Compute

[Compute instances](#)
[Compute clusters](#)
[Kubernetes clusters](#)
[Attached computes](#)

[+ New](#)
[Refresh](#)
[Delete](#)
[Edit columns](#)
[Reset view](#)
[View quota](#)

Name	☆	State	Size
batch-cluster		✓ Succeeded (0 nodes)	STANDARD_DS3_V2
cpu-cluster		✓ Succeeded (0 nodes)	STANDARD_DS3_V2

This section demonstrated use of a pipeline to deploy a trained model to a managed online or managed batch endpoint in Azure Machine Learning.

The single-environment deployment of this MLOps solution accelerator is complete. See the next section for information on adapting this pattern to your use case and broader MLOps practices.

MLOps Next Steps

To adapt this pattern to your use case and code, a guide to modifying files and pipelines in the ML project repo is below:

/data

In the `/data` directory, you can place your data to be registered with Azure ML. A `data.yml` file describing the dataset is used by the training pipeline in `/mlops/azureml/train/` and should be modified as needed for your data. Likewise, the training pipeline should be modified to refer to your dataset.

/data-science/src

In this directory you will modify, add, or remove code for your data science workflow.

/data-science/environment

In this directory, modify `train-conda.yml` to define the python environment required by your model training.

/infrastructure

This directory contains the infrastructure template and infrastructure pipeline for your Azure ML environment. In general, it should not need modification but review by your IT and verification you can create the defined resources is recommended.

/mlops/azureml/train

This directory contains the Azure ML yaml definitions for your dataset (`data.yml`), your python environment (`train-env.yml`), and the Azure ML training pipeline itself (`pipeline.yml`). Modify these as necessary to correctly refer to your data, environment, and python code steps as needed.

/mlops/azureml/deploy/batch and /mlops/azureml/deploy/online

These directories contain the yaml definitions and Azure ML pipelines for deployment of your model endpoints. Modify these as needed for your endpoint and model type.

/devops-pipelines

This directory contains the Azure DevOps pipeline definitions for deployment of Azure ML model training and endpoint pipelines. In general, these should need minimal changes except for updating references to data and training pipelines.

Troubleshooting

The following issues are commonly encountered on a fresh Azure DevOps organization.

"The repository taxi-fare-regression-template could not be retrieved"

This is a **repository checkout/authorization error before scripts run**, not an Azure ML training error. The initializer cannot import a missing source after Azure DevOps has already rejected its YAML repository reference.

1. Open **Repos** in the exact project selected by `adoProjectName` . Confirm the repaired source repository exists there with the exact selected name and a populated `main` branch. If missing, follow [Import the taxi regression template](https://github.com/alvinea28/taxi-fare-regression.git) using <https://github.com/alvinea28/taxi-fare-regression.git> and the Azure Repos name **taxi-fare-regression-template**. Authenticate only if the source requires it. Do not import it into the empty application target.
2. Confirm the target **taxi-fare-regression-demo** already exists and is still empty apart from its initial README. The initializer fills this target; it does not replace the working source project or create the repository object.
3. In **Project settings** → **Repositories** → **Security**, give the project Build Service **Read** on the accelerator and source. Give it **Read**, **Contribute**, and **Create branch** on the new target only. Use the current project's scoped Build Service identity; do not grant organization-wide administrator rights.
4. Authorize the referenced source and target for **this initializer pipeline** using **View/Permit** when Azure DevOps requests repository access. Pipeline authorization is separate from the repository's Git permissions. Keep repository-protection and project-scoped job authorization enabled; do not grant every pipeline access as a workaround.
5. Synchronize the latest accelerator `main` into Azure Repos and queue a new initialization run. Changing the GitHub copy or selecting the branch named `main` alone does not update an old Azure Repos import.

For the intended workflow, the missing item is often the reusable source import, not the manually created empty destination. After successful initialization, expect project files in the target plus four pipeline definitions whose first runs are disabled. Training and Azure infrastructure deployment are separate actions.

"No hosted parallelism has been purchased or granted"

By default, new Azure DevOps organizations have **zero** parallel jobs on Microsoft-hosted agents. The infrastructure pipeline will queue indefinitely until a free grant is approved.

Resolution: Submit the free-tier parallelism request form: <https://aka.ms/azpipelines-parallelism-request>. Approval is typically granted within 2–3 business days. If you need to deploy immediately, configure a [self-hosted agent](#).

"TF402455: Pushes to this branch are not permitted" or pipelines never appear

The user account creating the pipelines (typically the project administrator running `initialise-project`) must have **Create build pipeline** permission on the project.

Resolution: In Azure DevOps, navigate to **Project Settings** → **Permissions** → **Contributors** (or your group), and ensure **Create build pipeline** is set to *Allow*.

"A pipeline with name `deploy-...` already exists" on retry

If the `initialise-project` pipeline fails partway through and you re-run it, the second run will fail because pipelines and folders from the first attempt still exist.

Resolution: Inspect which step failed before retrying. If the project files were already committed but pipeline registration failed, preserve that generated repository and correct/register the missing pipeline definitions separately. The initializer intentionally refuses to overwrite an existing application. For a genuinely new attempt, use a different empty target repository. Do not delete your working taxi repository, source template, or Azure resources to fix an initialization or repository-authorization error.

"No image label found to route agent pool Azure Pipelines. Pool: Azure Pipelines, Image: ubuntu-20.04"

`ubuntu-20.04` was retired from the Microsoft-hosted agent pool. This affects older versions of `mlops-project-template` cloned before May 2026.

Resolution: Update to the latest `mlops-project-template` (uses `ubuntu-latest`), or in your project repo replace `vmImage: ubuntu-20.04` with `vmImage: ubuntu-latest` in all `mlops/devops-pipelines/*.yaml` and `infrastructure/.../*.yaml` files.

`az pipelines create` fails with "Could not queue the build" during `initialise-project`

The default agent queue association may not be established on a brand-new project.

Resolution: The `create_ado_pipelines.sh` script now resolves the queue ID dynamically. If you use a self-hosted pool, set `AGENT_POOL_NAME` in the `initialise-project` pipeline variables before running.

Next Steps in MLOps

This guide illustrated using Azure DevOps pipelines and Azure Machine Learning pipelines to adopt training automation, deployment, and repeatability for your data science workflow for a single Azure ML environment. Follow on MLOps practices may include the following:

- By default, the Azure DevOps pipelines in this accelerator do not execute unless manually triggered. This is to avoid unnecessary automatic runs during initial deployment of the accelerator. However, you may want to modify the pipelines to enhance automation. A few examples:
 - Modify the deployed [Azure ML model training pipeline to run on a schedule](#)
 - Modify the manual trigger on the [Azure DevOps deploy-model-training-pipeline to trigger on a schedule](#)
 - Modify the Azure DevOps model deployment pipeline to [trigger upon successful completion of the model training pipeline](#) instead of a manual trigger
- Trunk-based development using development and test/staging branches in Azure DevOps. Development and Production environments are typically separated. You may create an additional Dev branch in your project repo and deploy a development Azure ML environment from that. When code and model development in that environment is satisfactory, code and pipeline changes can be merged into the main branch and updates deployed to Prod. A `config-infra-dev.yml` example is provided in the project repo. To create this environment, repeat the steps in this guide from a dev branch in your ML project repo. Note that you should create a matching `dev` pipeline environment in Azure DevOps and new Azure service principal for an **Azure-ARM-Dev** service connection.
- Future work on this MLOps solution accelerator will include:
 - Integration of the [Azure Machine Learning registries](#) allowing you to register models and other artifacts from your development environment then deploying them in production directly.
 - Azure Machine Learning data drift monitoring
 - Integrated feature store
 - Deployment of secured Azure ML environments in a vnet

For questions, problems, or feature requests about this personal copy, please [submit an issue in the own repository](#).